Day 1: Introduction to Computer Vision

Welcome to the first lab session!
Today we will cover:

- What is Computer Vision?
- Images as Data
- Color Spaces
- Basic Transformations (overview)
- Filtering (intro)
- Applications

What is Computer Vision?

Computer Vision is the field of AI that enables machines to interpret and understand visual information. Applications include:

- Self-driving cars
- Medical imaging
- Surveillance
- Augmented Reality

```
import cv2
import matplotlib.pyplot as plt

# Load image
img = cv2.imread('sample.jpg')  # replace with your image file
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

print("Image shape:", img.shape)  # (height, width, channels)

# Display image
plt.imshow(img_rgb)
plt.title("Original Image")
```

```
Image shape: (5472, 3648, 3)
```



Images as Data

- Digital images are grids of pixels.
- Each pixel has intensity values (RGB channels).
- Resolution defines detail.

```
# Inspect pixel values
print("Pixel at (50,50):", img[50,50])  # BGR values
```

```
Pixel at (50,50): [172 169 161]
```

Color Spaces

- RGB: Red, Green, Blue
- HSV: Hue, Saturation, Value Converting between color spaces helps in tasks like color detection.

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV Color Space")
plt.axis('off')
plt.show()
```



Transformations Overview

- Translation: shifting image
- Scaling: resizing
- Rotation: turning image around a pivot
- Perspective: warping viewpoint

Filtering & Convolutions

- Convolution = sliding a kernel over an image.
- Filters detect edges, textures, and patterns.
- This is the foundation of CNNs.

```
import numpy as np

kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharp = cv2.filter2D(img, -1, kernel)

plt.imshow(cv2.cvtColor(sharp, cv2.COLOR_BGR2RGB))
plt.title("Sharpened Image")
plt.axis('off')
plt.show()
```




```
edges = cv2.Canny(img, 100, 200)

plt.imshow(edges, cmap='gray')
plt.title("Edge Detection")
plt.axis('off')
plt.show()
```



Applications of Computer Vision

- Traffic monitoring (detect cars)
- Healthcare (tumor detection)
- Retail (face recognition, product tracking)
- Creative AI (art generation)

Mini Project: Image Viewer

Build a simple viewer that:

- Loads an image
- Converts to grayscale
- Applies edge detection Try toggling between modes!

```python
def image_viewer(img_path, mode="original"):
    img = cv2.imread(img_path)
    if mode == "gray":
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    elif mode == "edges":
        img = cv2.Canny(img, 100, 200)
    plt.imshow(img, cmap='gray' if mode!="original" else
None)
    plt.title(f"Mode: {mode}")
    plt.axis('off')
    plt.show()

# Try different modes
image_viewer('sample.jpg', mode="original")
image_viewer('sample.jpg', mode="gray")
```




Mode: gray
PANTONE process guide uncoated
PANTONE solid to process guide coated
PANTONE process guide coated


Mode: edges

Day 2: Image Transformations

Today we will cover:

- Translation
- Scaling (Resizing)
- Rotation
- Affine Transformations
- Perspective Transformations

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Load image
img = cv2.imread('sample.jpg')   # replace with your image
file
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')
plt.show()
```



Translation

Shifting the image along the x or y axis.

```
M = np.float32([[1,0,100],[0,1,50]])  # shift right 100px, down 50px
translated = cv2.warpAffine(img, M, (cols, rows))

plt.imshow(cv2.cvtColor(translated, cv2.COLOR_BGR2RGB))
plt.title("Translated Image")
plt.axis('off')
```



Scaling (Resizing)

Resizing the image by a factor.

```
scaled = cv2.resize(img, None, fx=0.5, fy=0.5,
interpolation=cv2.INTER_LINEAR)

plt.imshow(cv2.cvtColor(scaled, cv2.COLOR_BGR2RGB))
plt.title("Scaled Image (50%)")
plt.axis('off')
```



Rotation

Rotating the image around its center.

```
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)  # rotate 45 degrees
rotated = cv2.warpAffine(img, M, (cols, rows))

plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image (45°)")
plt.axis('off')
```



Affine Transformation

Preserves parallelism of lines, defined by mapping three points.

```
pts1 = np.float32([[50,50],[200,50],[50,200]])
pts2 = np.float32([[10,100],[200,50],[100,250]])
M = cv2.getAffineTransform(pts1, pts2)
affine = cv2.warpAffine(img, M, (cols, rows))

plt.imshow(cv2.cvtColor(affine, cv2.COLOR_BGR2RGB))
plt.title("Affine Transformation")
plt.axis('off')
plt.show()
```



Perspective Transformation

Maps a quadrilateral to another quadrilateral.
Useful for document scanning.

```
pts1 = np.float32([[56,65],[368,52],[28,387],[389,390]])
pts2 = np.float32([[0,0],[300,0],[0,300],[300,300]])
M = cv2.getPerspectiveTransform(pts1, pts2)
perspective = cv2.warpPerspective(img, M, (300,300))

plt.imshow(cv2.cvtColor(perspective, cv2.COLOR_BGR2RGB))
plt.title("Perspective Transformation")
plt.axis('off')
plt.show()
```



Mini-Project: Image Augmentation Tool

Build a simple tool that applies transformations:

- Translate
- Scale
- Rotate
- Perspective

```python
def transform_image(img_path, mode="original"):
    img = cv2.imread(img_path)
    rows, cols = img.shape[:2]

    if mode == "translate":
        M = np.float32([[1,0,100],[0,1,50]])
        img = cv2.warpAffine(img, M, (cols, rows))
    elif mode == "scale":
        img = cv2.resize(img, None, fx=0.5, fy=0.5)
    elif mode == "rotate":
        M = cv2.getRotationMatrix2D((cols/2, rows/2), 45,
1)
        img = cv2.warpAffine(img, M, (cols, rows))
    elif mode == "perspective":
        pts1 = np.float32([[56,65],[368,52],[28,387],
[389,390]])
        pts2 = np.float32([[0,0],[300,0],[0,300],
[300,300]])
        M = cv2.getPerspectiveTransform(pts1, pts2)
        img = cv2.warpPerspective(img, M, (300,300))

    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(f"Mode: {mode}")
    plt.axis('off')
    plt.show()

# Try different modes
transform_image('sample.jpg', mode="translate")
```

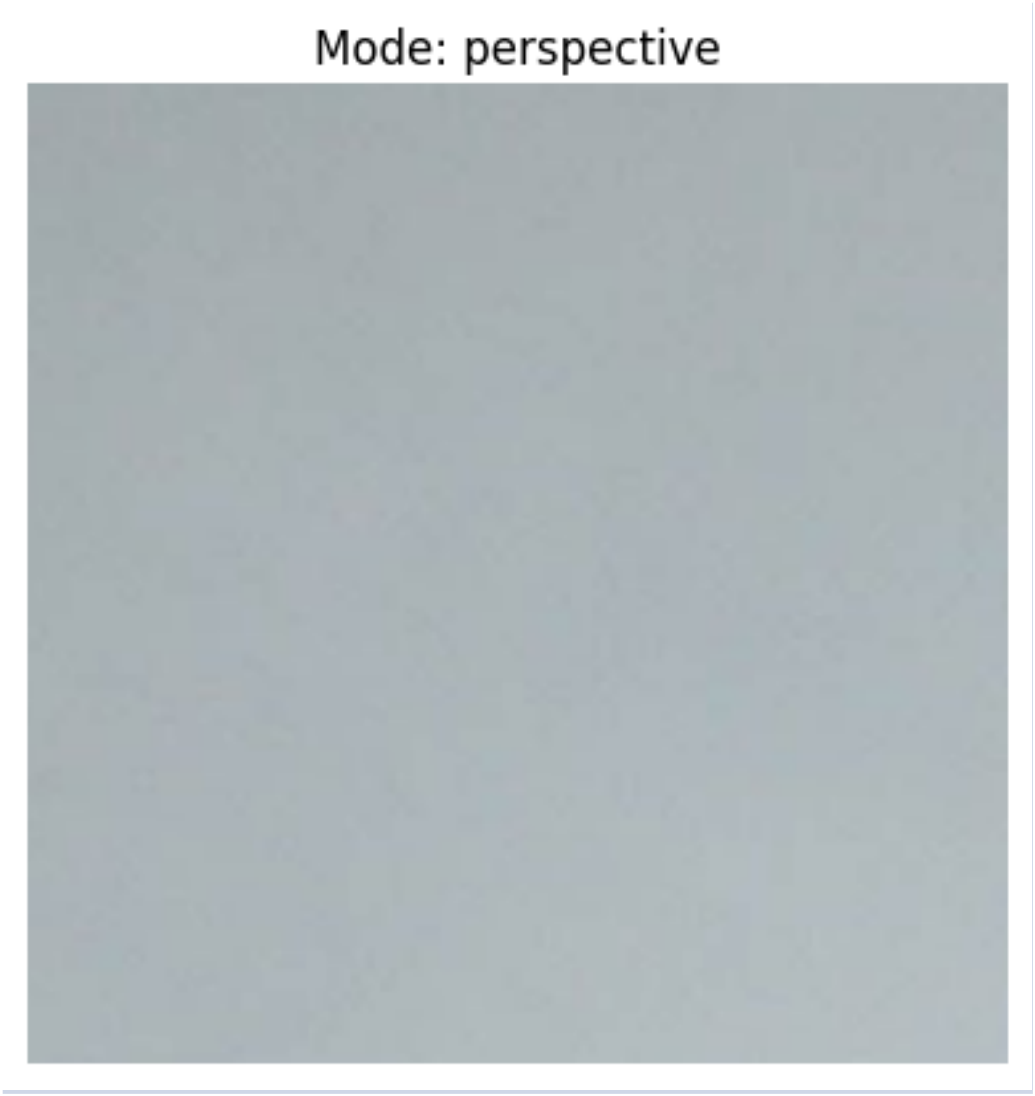

Mode: translate
PANTONE process guide uncoated
PANTONE solid to process guide coated
PANTONE process guide coated

Mode: scale


PANTONE
process guide
uncoated
PANTONE
solid to process guide
coated
process guide
coated

Mode: rotate


PANTONE process guide uncoated
PANTONE solid to process guide coated
NTONE process guide coated


Mode: perspective

Day 3: Filtering & Convolutions

Today we will cover:

- Why Filtering Matters
- Convolution Basics
- Common Filters (Blur, Sharpen, Edge Detection)
- Edge Detection (Canny)
- Applications
- Lab Exercise
- Mini-Project

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# Load image
img = cv2.imread('sample.jpg')   # replace with your image file
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')
```



Why Filtering Matters

- Enhances or suppresses features.
- Used for blurring, sharpening, and edge detection.
- Foundation of Convolutional Neural Networks (CNNs).

Convolution Basics

- Kernel = small matrix applied to an image.
- Sliding window computes weighted sum.
- Produces a feature map highlighting patterns.

```
kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])  # sharpening kernel
sharp = cv2.filter2D(img, -1, kernel)

plt.imshow(cv2.cvtColor(sharp, cv2.COLOR_BGR2RGB))
plt.title("Sharpened Image")
plt.axis('off')
```



Common Filters

- Blur (average, Gaussian)
- Sharpen
- Edge detection (Sobel, Laplacian)

```
blur = cv2.GaussianBlur(img, (5,5), 0)
edges_sobel = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(blur, cv2.COLOR_BGR2RGB))
plt.title("Gaussian Blur")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(edges_sobel, cmap='gray')
plt.title("Sobel Edge Detection")
plt.axis('off')

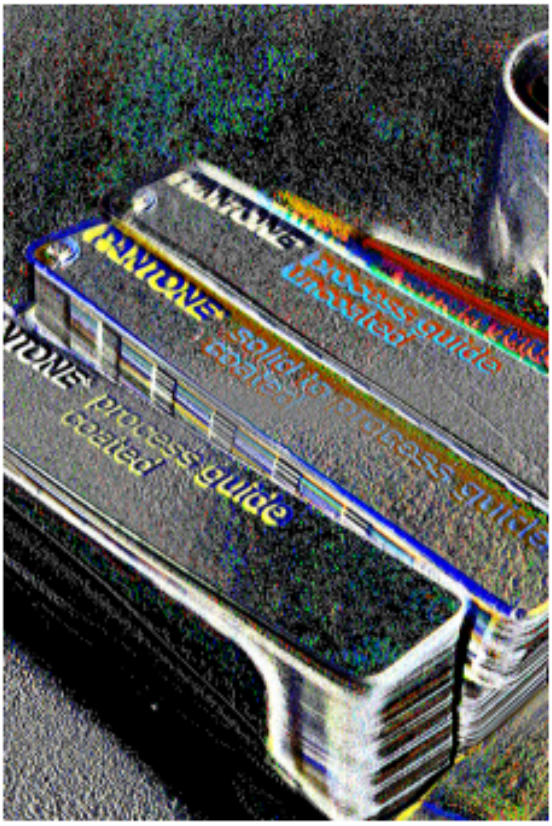
plt.show()
```

```
Clipping input data to the valid range for imshow with RGB
```

Gaussian Blur



Sobel Edge Detection



Canny Edge Detection

- Combines gradient + thresholding.
- Produces clean edge maps.

```
edges_canny = cv2.Canny(img, 100, 200)

plt.imshow(edges_canny, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis('off')
plt.show()
```



Applications of Filtering

- Medical imaging (tumor boundaries)
- Document scanning
- Autonomous driving (lane detection)

Lab Exercise

- Apply blur, sharpen, and edge detection.
- Compare outputs side by side.

```
filters = {
    "blur": cv2.blur(img, (5,5)),
    "sharpen": cv2.filter2D(img, -1, kernel),
    "edges": cv2.Canny(img, 50, 150)
}

for name, output in filters.items():
    plt.figure()
    plt.imshow(output if name=="edges" else
cv2.cvtColor(output, cv2.COLOR_BGR2RGB),
               cmap='gray' if name=="edges" else None)
    plt.title(name.capitalize())
    plt.axis('off')
```



Sharpen


PANTONE
process guide
uncoated
PANTONE
solid to process guide
coated
PANTONE
process guide
coated



Mini-Project: Edge-Based Object Detector

- Input: photo of simple shapes.
- Output: contours + labels.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

img_contours = img.copy()
for i, cnt in enumerate(contours):
    x,y,w,h = cv2.boundingRect(cnt)
    cv2.rectangle(img_contours, (x,y), (x+w,y+h),
(0,255,0), 2)
    cv2.putText(img_contours, f"Obj{i+1}", (x,y-10),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1)

plt.imshow(cv2.cvtColor(img_contours, cv2.COLOR_BGR2RGB))
```

